CS430/910: Foundations of Data Analytics

Classification 2 | Dr Ayse Sunar

Objectives

- To understand the concept of classifiers and their construction
- See how decision tree classifiers are constructed
- Statistical classifiers: Naive Bayes
- Linear separation via SVM classifiers
- Construct and compare classifiers using Weka

Part B: Classification: Introduction 2

Class Imbalance

- Consider a classifier which aims to predict a certain type of cancer, which is absent in 98% of the population.
- The classifier can predict *positive* or *negative* based on some input data.
- We could really easily get a 98% accuracy rate just by predicting *negative*!
- Although, this is not acceptable, as we would miss the 2% of people who do have cancer…

Class Imbalance

Definition

- Class imbalance describes the situation where the classes present within the data are not equally represented.
- Can occur when there are only two classes (see cancer example), but can also occur in multi class examples (where one or more classes are much rarer than others in the dataset).

Class Imbalance

Solutions

1. True Positive Rate (Sensitivity) and True Negative Rate (Specificity) report the proportion of positive examples correctly identified, and the proportion of negative examples correctly identified, respectively.
2. Oversampling
3. Undersampling
4. Threshold Moving
5. Ensemble Techniques

Class Imbalance

Oversampling

- Consider a training data set which contains 500 positive examples and 5000 negative examples…
- **Class Imbalance** - Model can just always predict negative to get a fairly high accuracy rate.
- **Oversampling** - Replicate the examples of the rarer class to form a new training set with 5000 positive examples and 5000 negative examples.
- Could use data augmentation rather than simply replicating existing data.

Class Imbalance

Undersampling

- Consider a training data set which contains 500 positive examples and 5000 negative examples…
- **Class Imbalance** - Model can just always predict negative to get a fairly high accuracy rate.
- **Undersampling** - Randomly eliminate the negative examples until the number of negative examples is equal to the number of positive examples (in our example, 500 positive and 500 negative).

Class Imbalance

Threshold-Moving

- Recall a few slides ago, we mentioned a classifier that returned an output value [0,1], and the instance was considered positive if the output value was $\geq t$.
- Consider $t = 0.98$. Difficult for example to be considered positive as output value would need to be≥ 0.98.
- Threshold-moving involves moving the value of the threshold, t, to make it easier for rarer classes to be classified.

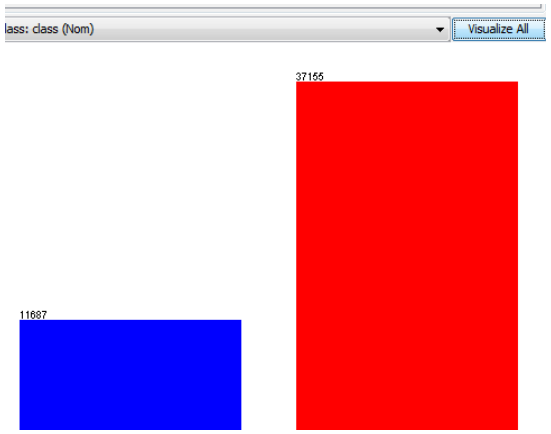
Class Imbalance

Ensemble Techniques

- Combines multiple models (M_1,M_2,…M_k), with the aim of creating an improved, composite classification model, $M*$.
- Individual classifiers within the ensemble may include version of the approaches described previously (oversampling, undersampling, threshold moving).

Majority Class Classifier

- A trivial baseline: always predict the majority class
  - More generally, predict the distribution of the class values
- Suppose the majority class occurs a p fraction of the time
  - Then should achieve (approximately) p accuracy
- Not a useful or meaningful classifier
  - But an important benchmark: any good model should clearly outperform the majority class classifier
- Implemented as “ZeroR” model in Weka: **classifiers.rules.ZeroR**
  - Generalises to numeric data: predicts mean



Weka Results for Majority Class Classifier

```
Relation:     adult
Instances:    48842
Attributes:   15
Test mode:split 66.0% train, remainder test
=== Classifier model (full training set) ===
ZeroR predicts class value: <=50K
Time taken to build model: 0.01 seconds
=== Evaluation on test split ===
Correctly Classified Instances       12728               76.647  %
Incorrectly Classified Instances      3878               23.353  %
Kappa statistic                          0
Mean absolute error                      0.3626
Root mean squared error                  0.4232
Relative absolute error                 100      %
Root relative squared error             100      %
Total Number of Instances            16606
=== Detailed Accuracy By Class ===
               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0         0          0         0         0          0.5      >50K
                 1         1          0.766     1         0.868      0.5      <=50K
Weighted Avg.    0.766     0.766      0.587     0.766     0.665      0.5
=== Confusion Matrix ===
     a     b   <-- classified as
     0  3878 |     a = >50K
     0 12728 |     b = <=50K
```

Nearest Neighbour Classifier

- Assume that examples that are “close” should behave alike
  - To classify an example, find most similar training example
- k-nearest neighbour classifier: find the k closest examples
  - And predict the majority vote
  - N-nearest neighbour classifier: equivalent to Majority-class

Nearest Neighbour Classifier

- How to determine which is the closest?
  - Use an appropriate distance, as discussed earlier (L_1/ L_2 distance)
  - Finding the nearest neighbour can be slow: scan through all points
  - Some specialised data structures help, but still not fast

Nearest Neighbour Classifier

In Weka

- Listed as an “instance based classifier” (classifiers.lazy.IBk)
  - Class of classifiers that use training data as model
  - IB1: very simple 1-nearest neighbour classifier
  - IBk: more flexible k-nearest neighbour classifier
    - User provides k (default: 1)
    - Or can use cross-validation to pick best value of k
    - Pick search algorithm to use (default: linearscan)
    - Pick distance to use (default: Euclidean; L_1, L_∞ also available)
  - Fast to build (just look at data), but slow to evaluate/apply

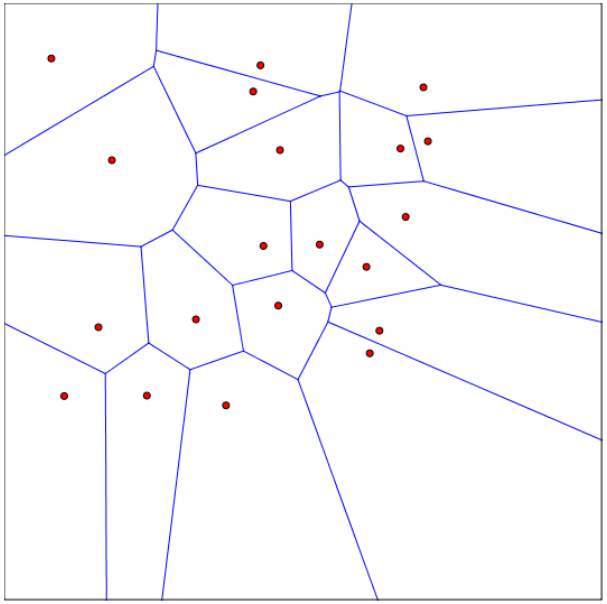
Weka Results for adult.data kNN

```
Relation:     adult
Instances:    48842
Attributes:   15
Test mode:split 66.0% train, remainder test
=== Classifier model (full training set) ===
IB1 instance-based classifier using 10 nearest neighbour(s) for classification
Time taken to build model: 0.11 seconds
=== Evaluation on test split ===
Correctly Classified Instances       13851               83.4096 %
Incorrectly Classified Instances      2755               16.5904 %
Kappa statistic                          0.5359
Mean absolute error                      0.2122
Root mean squared error                  0.3392
Relative absolute error                 58.5032 %
Root relative squared error             80.1583 %
Total Number of Instances            16606
=== Detailed Accuracy By Class ===
               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0.643     0.108      0.645     0.643     0.644      0.874    >50K
                 0.892     0.357      0.891     0.892     0.892      0.874    <=50K
Weighted Avg.    0.834     0.299      0.834     0.834     0.834      0.874
=== Confusion Matrix ===
     a     b   <-- classified as
  2492  1386 |     a = >50K
  1369 11359 |     b = <=50K
```

Part C: Decision Trees

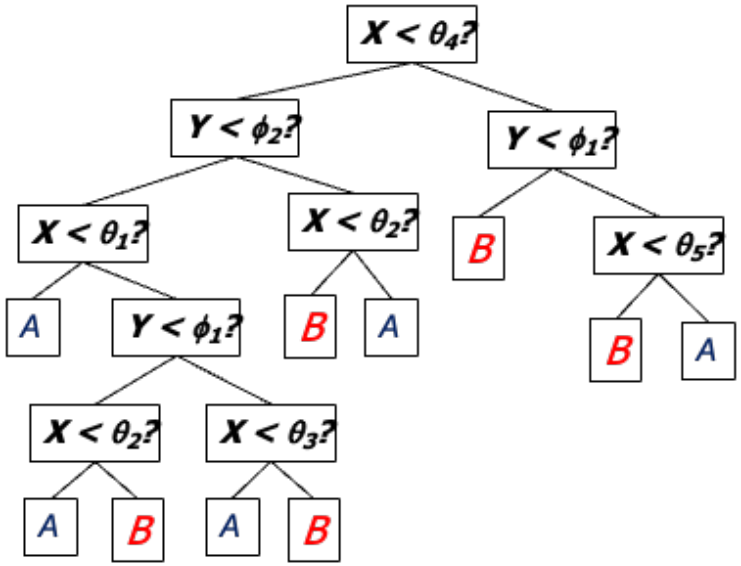
From Points to Regions

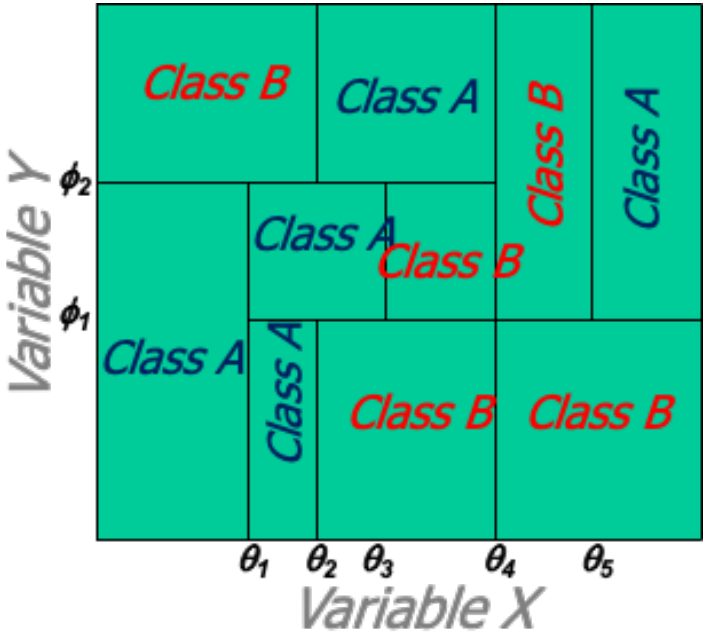
- The principle behind the nearest neighbour classifier is sound
  - Examples with similar class are “close” in the data space
- The partition of space induced by nearest neighbours is complex
  - Difficult to search/index
  - Very detailed description
  - Technically a voronoi diagram
- Other classifiers aim for a simpler description of the space:
  - Divide space into “rectangles”
  - Divide hierarchically for easy indexing



Partition of Space

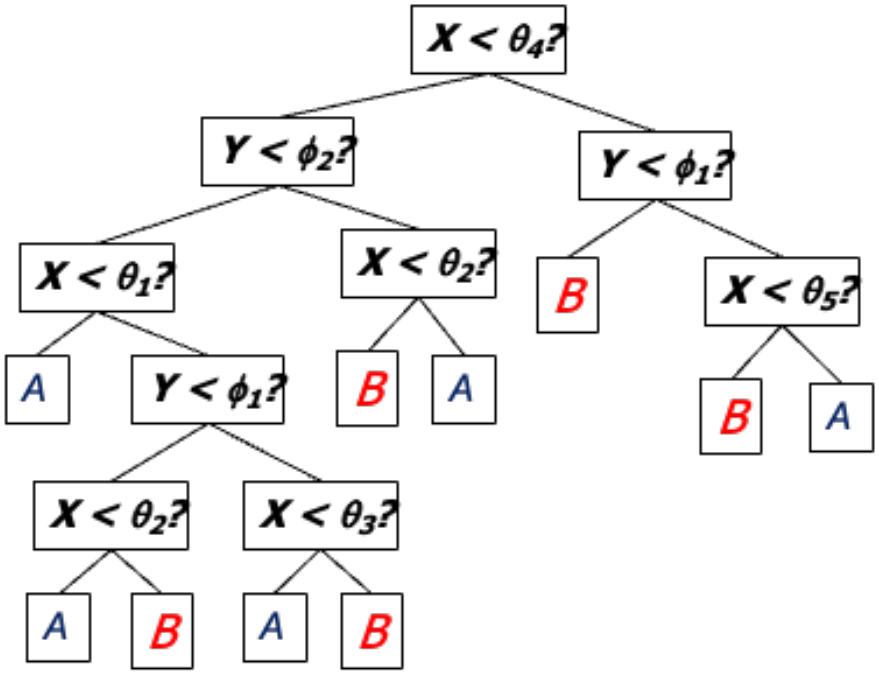
- Hierarchical divisions of space are easy to navigate (for classification)
  - Can be represented as a tree





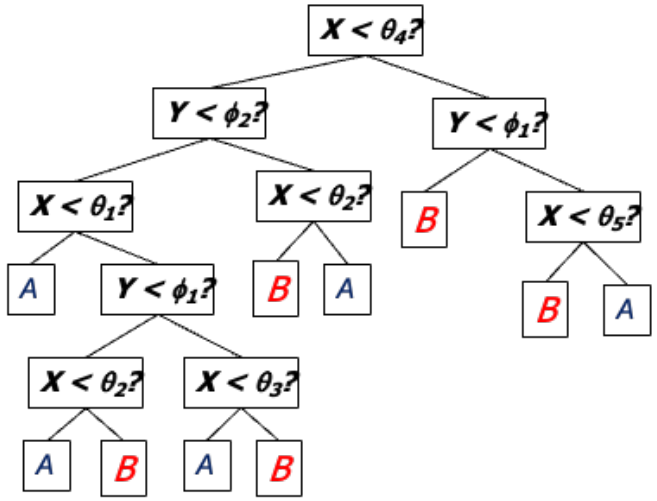
Partition of Space

- A *Decision Tree*:
  - Convention:
    - True to left, False to right
    - Many ways to construct



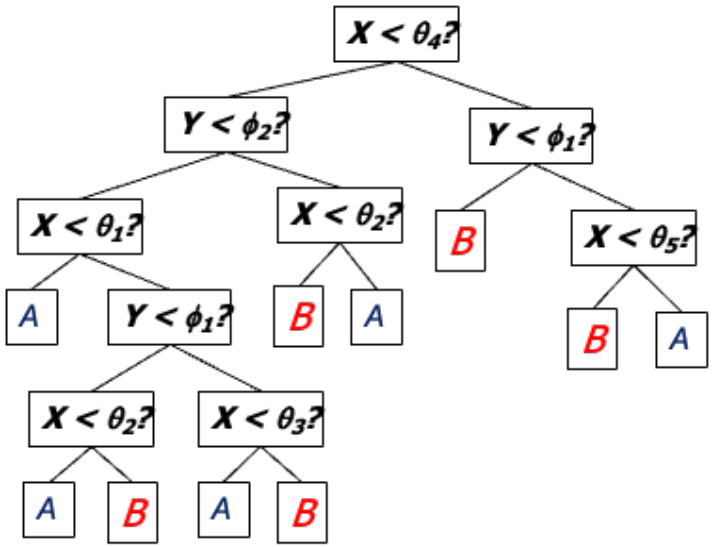
Building a Decision Tree

- Some definitions:
  - **Internal node**: A non-leaf node which denotes a test on an attribute.
  - **Leaf node** (or **terminal node**): Holds a class label.
  - **Root node**: Topmost node.
- For a given example, we test its attribute values against the decision tree.
  - Trace a path to the leaf node to determine the class.



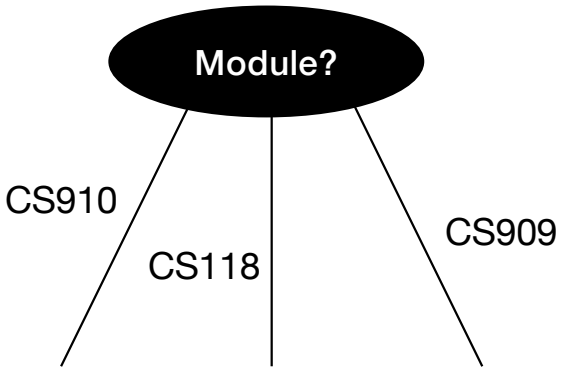
Building a Decision Tree

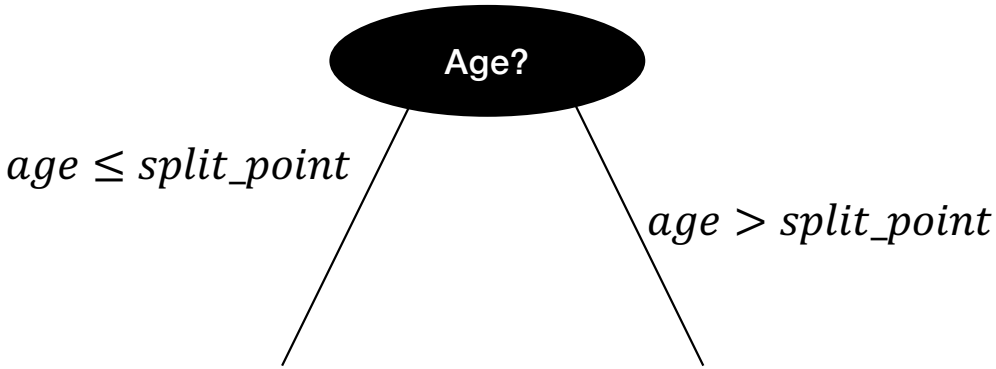
- Many choices in building a decision tree
  - Start with all examples at ‘root’ of tree
  - Choose an attribute to partition on, and a split value
  - Recursively apply the algorithm on both halves
  - Continue splitting until some terminating condition is met:
    - All examples in a partition belong to the same class
    - There is only one example in the partition
  - Label each ‘leaf’ of the tree with the majority class in the leaf
    - Or with the class distribution
- Main difference between algorithms is **how to choose splits**



Building a Decision Tree

Choice of Splits





Module is discrete-valued

Age is continuous-valued

Building a Decision Tree

Choice of Splits

- **Binary attributes** (e.g. Yes/No): only one way to split!
- **Categoric attributes**, e.g. {Single, Married, Widowed, Divorced}
  - Each node could have multiple children (depending on attribute selection measure)
  - Could pick one attribute value versus rest to split on
    - n choices if attribute has n possibilities
  - Could pick any subset of attribute values to split into two
    - ~2^{n-1} choices if attribute has n possibilities
      - e.g. {S}, {S, M}, {S, W}, {S, D}, {S, M, W}, {S, M, D}, {S, W, D}

Building a Decision Tree

Choice of Splits

- **Ordered attributes**, e.g. Grade = (A, B, C, D, E)
  - Each node could have multiple children (depending on attribute selection measure)
  - Or, pick some point to make a split into two.

How to Choose Splits

Attribute Selection Measures

- An attribute selection measure is a heuristic for selecting the best splitting criterion, i.e. that which best separates a given data partition, D, of class-labeled training tuples (examples) into individual classes.
- Ideal scenario: We divide D into smaller partitions according to the outcomes of the splitting criterion, and each partition (at leaf level) would be pure.
  - **Pure**: All the tuples that fall into the given partition would fall into the same class.
- Attribute selection measures are sometimes referred to as *splitting rules*.

How to Choose Splits

Attribute Selection Measures

- Attribute selection measures provide a ranking for each attribute describing the given training tuples.
- The attribute having the “best” score (highest or lowest depending on the method) is chosen as the splitting attribute for the given tuples.
- Three popular attribute selection measures:

    1. **Information Gain**
    2. **Gain Ratio**
    3. **Gini Index**

How to Choose Splits

Notation

- D = Data Partition (training set of class-labelled tuples).
- Let the class label have m distinct classes, C_i (for $i = 1, \ldots, m$).
- Let $C_{i,D}$ be the set of tuples of class C_i in D.
- $|D|$ = Number of tuples in D.
- $|C_{i,D}|$ = Number of tuples in $C_{i,D}$.

Information Gain

- The attribute with the highest information gain is chosen as the splitting attribute for node N, for a data partition D.
  - Goal is to minimise the information needed to classify the tuples in the resulting partitions.
- The expected information needed to classify a tuple in D is given by:

$$Info(D) = -\sum_{i=1}^{m} p_i log_2(p_i)$$

where p_i is the nonzero probability that an arbitrary tuple in D belongs to class C_i, and is estimated by $\frac{|C_{i,D}|}{|D|}$.

Information Gain

- A log function to the base 2 is used, because the information is encoded in bits.
- $Info(D)$ is the average amount of information needed to identify the class label of a tuple in D, and is also known as the *entropy* of D.
- Say we were to partition the tuples in D on some attribute A having v distinct values, $\{a_1, a_2, \ldots, a_v\}$.
  - If A is discrete-valued, attribute A can be used to split D into v partitions, $\{D_1, D_2, \ldots, D_v\}$, where D_j contains the tuples in D that have outcome a_j of A. These partitions would correspond to the branches grown from node N.

Information Gain

- Ideally, these partitions would be pure (i.e. all the examples that fall into the given partition would fall into the same class), but this is **unlikely**.
- How much more information would we need after the previous partitioning to arrive at an exact classification? This is measured by:

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

- $Info_A(D)$ is the expected information required to classify a tuple from D based on the partitioning from A.
  - The smaller the information still required, the greater the purity of the partitions.

Information Gain

- Information Gain is the difference between the two:

$$Gain(A) = Info(D) - Info_A(D)$$

- Higher value implies that $Info_A(D)$ is smaller, hence we choose attribute with the highest $Gain(A)$.

Information Gain

- D = Data Partition (training set of class-labelled tuples).
- $m = 2$, (plays_sports has 2 classes, C_1 and C_2).
- $|D|$ = Number of tuples in D = 6.
- Remember, $C_{i,D}$ is the set of tuples of class C_i in D,$|D|$ is the number of tuples in D, and $|C_{i,D}|$ is the number of tuples in $C_{i,D}$.

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Information Gain

- Expected information needed to classify a tuple in D:

$$Info(D) = -\sum_{i=1}^{m} p_i log_2(p_i)$$

$$= -\frac{4}{6}log_2\left(\frac{4}{6}\right) - \frac{2}{6}log_2\left(\frac{2}{6}\right)$$

$$= 0.9183 \text{ bits}$$

Where p_i us the probability that an arbitrary tuple in D belongs to C_i (estimated by $\frac{|C_{i,D}|}{|D|}$).

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Information Gain

- Expected information requirement for *income*:

$$Info_{income}(D) = = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{2}{6} \times \left(-\frac{1}{2} log_2\left(\frac{1}{2}\right) - \frac{1}{2} log_2\left(\frac{1}{2}\right)\right) +$$

$$\frac{1}{6} \times \left(-\frac{1}{1} log_2\left(\frac{1}{1}\right)\right) + \frac{3}{6} \times \left(-\frac{2}{3} log_2\left(\frac{2}{3}\right) - \frac{1}{3} log_2\left(\frac{1}{3}\right)\right)$$

$$= 0.7925 \text{ bits}$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Information Gain

- The gain in information from partitioning on income would be:

$$
\begin{aligned}
Gain(income) &= Info(D) - Info_{income}(D) \\
&= 0.9183 - 0.7925 \\
&= 0.1258 \text{ bits}
\end{aligned}
$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Information Gain

- Expected information requirement for $is_student$:

$$Info_{is_student}(D) = = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

$$= \frac{4}{6} \times \left(-\frac{2}{4} log_2\left(\frac{2}{4}\right) - \frac{2}{4} log_2\left(\frac{2}{4}\right)\right) +$$

$$\frac{2}{6} \times \left(-\frac{2}{2} log_2\left(\frac{2}{2}\right)\right)$$

$$= 0.66666 \text{ bits}$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Information Gain

- The gain in information from partitioning on is_student would be:

$$Gain(is_student) = Info(D) - Info_{is_student}(D)$$
$$= 0.9183 - 0.6666$$
$$= 0.2517 \text{ bits}$$

- $is_student$ has the highest Information Gain, so it is chosen as the splitting attribute.

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Decision Tree Algorithms and Split Rules

- Different decision tree algorithms use different split rules:
  - ID3 uses **Information Gain**
  - C4.5 uses **Gain Ratio**
  - CART (classification and regression tree) uses the **Gini Index**

Gain Ratio

- Problem with Information Gain: It is biased towards tests with many outcomes.
  - Consider an attribute user_ID. Splitting on user_ID would result in a large number of partitions (equal to number of values), each containing one tuple.
    - Each partition would be pure, so Information Gain would prefer this outcome!
- However, this partition is useless…

Gain Ratio

- Gain Ratio is an extension of Information Gain, which overcomes this problem.
- It is used by C4.5, a successor of ID3.
- Gain Ratio is defined as:

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

Gain Ratio

- Gain Ratio is an extension of Information Gain, which overcomes this problem.
- It is used by C4.5, a successor of ID3.
- Gain Ratio is defined as:

Information Gain

$$GainRatio(A) = \frac{\boxed{Gain(A)}}{SplitInfo_A(D)}$$

Gain Ratio

- Gain Ratio is an extension of Information Gain, which overcomes this problem.
- It is used by C4.5, a successor of ID3.
- Gain Ratio is defined as:

$$GainRatio(A) = \frac{Gain(A)}{\boxed{SplitInfo_A(D)}}$$

Normalisation

Gain Ratio

- Gain Ratio is an extension of Information Gain, which overcomes this problem.
- It is used by C4.5, a successor of ID3.
- Gain Ratio is defined as:

$$GainRatio(A) = \frac{Gain(A)}{\boxed{SplitInfo_A(D)}}$$

Normalisation

How do we define $SplitInfo_A(D)$?

Gain Ratio

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times log_2(\frac{|D_j|}{|D|})$$

- Note that for each outcome $(1, \ldots, v)$, it considers the number of tuples having that outcome, with respect to the total number of tuples in D.
- The attribute with the **maximum Gain Ratio** is selected as the splitting attribute.

Gain Ratio

Example

- As shown previously:

$$Gain(is_student) = 0.2517 \text{ bits}$$

$$SplitInfo_{is_student}(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times log_2\left(\frac{|D_j|}{|D|}\right)$$

$$= -\frac{4}{6}log_2\left(\frac{4}{6}\right) - \frac{2}{6}log_2\left(\frac{2}{6}\right)$$

$$= 0.9183$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gain Ratio

Example

$$GainRatio(is_student) = \frac{Gain(is_student)}{SplitInfo_{is_student}(D)}$$
$$= \frac{0.2517}{0.9183}$$
$$= 0.2741$$

- The attribute with the **maximum** Gain Ratio is taken as the splitting attribute.

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Next lecture

- Gini Index
- How to prune a tree
- Naïve Bayes and SVM(?)

Acknowledgements

- G Watson [Warwick, CS430/CS910]
- Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.
- G Cormode [Warwick, CS910]
- F Ciucu [Warwick, CS430/CS910]